ForgeAgent: A Fully Local Autonomous Software-Engineering Agent

Verified Reporting, Policy-Gated Tool Execution and Crash-Safe State on Consumer Hardware with an Open-Weights Model

Anthony Efemena Edjenuwa
Independent researcher and developer
github.com/bahdleen/forgeagent

Technical report — 14 October 2026

Abstract

Autonomous coding agents are usually thin clients for hosted frontier models. This report describes *ForgeAgent*, a complete, open-source agent platform that runs entirely on one consumer laptop (an NVIDIA RTX 4090 Laptop GPU with 16 GB of VRAM and 64 GB of RAM) using a 27-billion-parameter open-weights model quantised to 4 bits and served by llama.cpp. The contribution is not a new model but the *systems engineering* that makes a small, locally hosted, imperfect model safe and useful enough to modify real code bases: (i) an LLM gateway that streams, cancels instantly, and *recovers* malformed tool calls; (ii) a typed tool layer with a three-mode permission policy, workspace confinement and an immutable audit log; (iii) an orchestration loop with planning, a dependency-aware task queue, role-specialised prompts that share one model instance, and guards that refuse to let the agent declare success while verification fails; (iv) crash-safe persistence so interrupted work can be resumed; and (v) context budgeting, compaction and a persistent, searchable project memory. The final report shown to the user is generated from the audit log rather than from the model's claims. On the target machine we measured throughput, GPU/CPU utilisation, tool-call reliability and executed-code correctness for three quantisations of the same model, and found that the quantisation that fits entirely in VRAM generated about five times faster (36 versus 7 tokens/s) with no difference we could detect on a small workload. We describe the architecture, give the key algorithms and real source excerpts, report automated-test results and end-to-end experiments with the actual local model, and document the engineering pitfalls we met (stale bytecode caches, a commit-after-response race, CRLF-incompatible patches, over-eager secret redaction). The report is written to be *educational*: every section lists learning objectives, highlights the underlying principle, and ends with exercises. We state limitations plainly, in particular that the system is not a sandbox.

Keywords: LLM agents, local inference, llama.cpp, tool use, software engineering agents, prompt injection, least privilege, context management.

Contents

1 Introduction **3**
1.1 Requirements 3
1.2 Contributions 3

2 Background **3**

3 Related work **4**

4 System architecture **4**
4.1 Hardware and model . . . 5
4.2 Roles on a single model . . . 5

5 The local LLM gateway **6**
5.1 Tool-call recovery . . . 6
5.2 Instant cancellation . . . 7

6 Typed tools and the permission policy **8**
6.1 Tool abstraction . . . 8
6.2 Workspace confinement . . . 8
6.3 Autonomy modes . . . 9
6.4 Command classification . . . 10

7 The agent loop **11**
7.1 Control flow . . . 11
7.2 Guards against bad agent behaviour . . . 12
7.3 Truthful reporting . . . 13
7.4 Role prompts . . . 13

8 Persistence, recovery and resume **13**

9 Context management **14**

10 Persistent project memory **16**

11 User interface **16**

12 Evaluation **16**
12.1 Automated tests . . . 17
12.2 End-to-end experiments with the real model . . . 17
12.3 Quantisation trade-offs: speed, memory and quality . . . 20

13 Engineering lessons **21**

14 Limitations and threats to validity **23**

15 Conclusion and future work **24**

A Reproducing the results **24**

B Exercises **25**

C Glossary **25**

1 Introduction

Learning objectives

After this section you should be able to: (1) define an *LLM agent* and its control loop; (2) explain why local deployment changes the engineering problem; (3) list the requirements ForgeAgent was built to satisfy.

A *language-model agent* wraps a model in a loop: the model proposes an action, an environment executes it, and the observation is fed back [25, 28]. For software engineering, the actions are file edits, shell commands and test runs; benchmarks such as SWE-bench [11] and agent–computer interfaces such as SWE-agent [27] show that carefully designed tools matter as much as the underlying model.

Most such systems call a remote API. Running the model *locally* offers privacy, no per-token cost, offline operation and full control, but it changes the engineering constraints in four ways:

1. **Weaker, less reliable models.** Small open models emit malformed tool calls, loop, or announce a plan and stop.
2. **Scarce memory.** A 27B-parameter model at 4 bits occupies about 15.4 GiB (Section 4.1); the context window competes with the weights for VRAM.
3. **Low throughput.** Partial CPU offload yields roughly 10 generated tokens per second, so wasted steps are expensive.
4. **Weaker alignment guarantees.** Some community models are deliberately modified to refuse less [1]; safety must therefore live in the *system*, not in the model.

1.1 Requirements

ForgeAgent was specified to: inspect and modify repositories; execute terminal commands and tests; operate a browser; plan and decompose work; keep long-running progress and resume after restarts; coordinate specialised roles on a single model instance; keep persistent memory; expose a professional UI; and respect explicit permission boundaries, with execution permissions kept separate from the model's own refusal behaviour (no keyword filters in the gateway).

1.2 Contributions

- A complete, runnable architecture (Section 4) with 28 typed tools and 5 logical roles on one model.
- A *tool-call recovery* layer making small models usable (Section 5).
- *Verified reporting*: guards and an audit-derived report that keep the agent honest (Section 7).
- A least-privilege permission model with workspace confinement, and an explicit, honest threat model (Section 6).
- Crash-safe persistence and resume, context budgeting and compaction, and local vector memory (Sections 8–10).
- An empirical evaluation with the real local model and a catalogue of engineering lessons (Sections 12–13).

2 Background

Learning objectives

Understand tool calling, quantisation and the KV cache well enough to reason about the hardware numbers later; understand prompt injection.

Tool calling. The de-facto wire format is the OpenAI chat-completions schema [14]: the client sends JSON-Schema tool definitions; the model returns `tool_calls` with a function name and a JSON string of arguments; the client executes them and replies with `tool` messages. llama.cpp [8] reproduces this interface for local models, applying the model's chat template server-side. Models differ in how reliably they follow it [21], which motivates Section 5.

Quantisation and memory. Post-training quantisation [6, 7] stores weights in 4–8 bits. llama.cpp's `Q4_K_M` format averages roughly 4.5–5 bits per weight. During generation the transformer [24] also keeps a *key–value (KV) cache* whose size grows linearly with context length; cache memory dominates serving cost at long contexts [12]. FlashAttention-style kernels [4] reduce attention memory traffic (enabled with `-fa`).

Long context is not free. Even when a model accepts a long prompt, it uses information in the middle of it poorly [13]. Agents therefore benefit from *short, curated* contexts, which motivates compaction (Section 9).

Prompt injection. When an agent reads files, web pages or tool output, any of them may contain text that tries to give the agent new instructions [9, 17]. Benchmarks such as AgentDojo [5] show that models alone do not reliably resist this. The practical defence is architectural: *limit what a hijacked agent can do* (least privilege [20]) and keep a record of what it did [15].

Key idea

Capability, not intention. A model's refusal behaviour is an unreliable control. ForgeAgent enforces permissions *in code*, outside the model, and treats everything the model reads as untrusted data.

3 Related work

ReAct [28] interleaves reasoning and acting and is the template for our loop. Reflexion [22] adds verbal self-feedback after failures; ForgeAgent's "error → fix" memory records (Section 10) play a similar role but are stored persistently and are human-editable. MemGPT [16] treats the context window like virtual memory, paging information in and out; our compaction and tool-output truncation with on-demand retrieval (`read_full_output`) follow the same intuition at a smaller scale. SWE-agent [27] demonstrates that a purpose-built tool interface (e.g. guarded edit commands) improves success; our `edit_file` returns the closest matching text on failure for the same reason. Surveys of LLM-based agents [25] situate these components. Unlike these research systems, ForgeAgent targets a complete *local product*: UI, persistence, permissions and installation are part of the contribution. We do not claim state-of-the-art benchmark performance and do not evaluate on SWE-bench.

4 System architecture

Learning objectives

Name the components of the system and the direction of every dependency; explain why roles share one model instance.

Figure 1 shows the system. A React/TypeScript front end communicates with a FastAPI [19] back end over REST and a WebSocket that carries live events. The back end persists everything

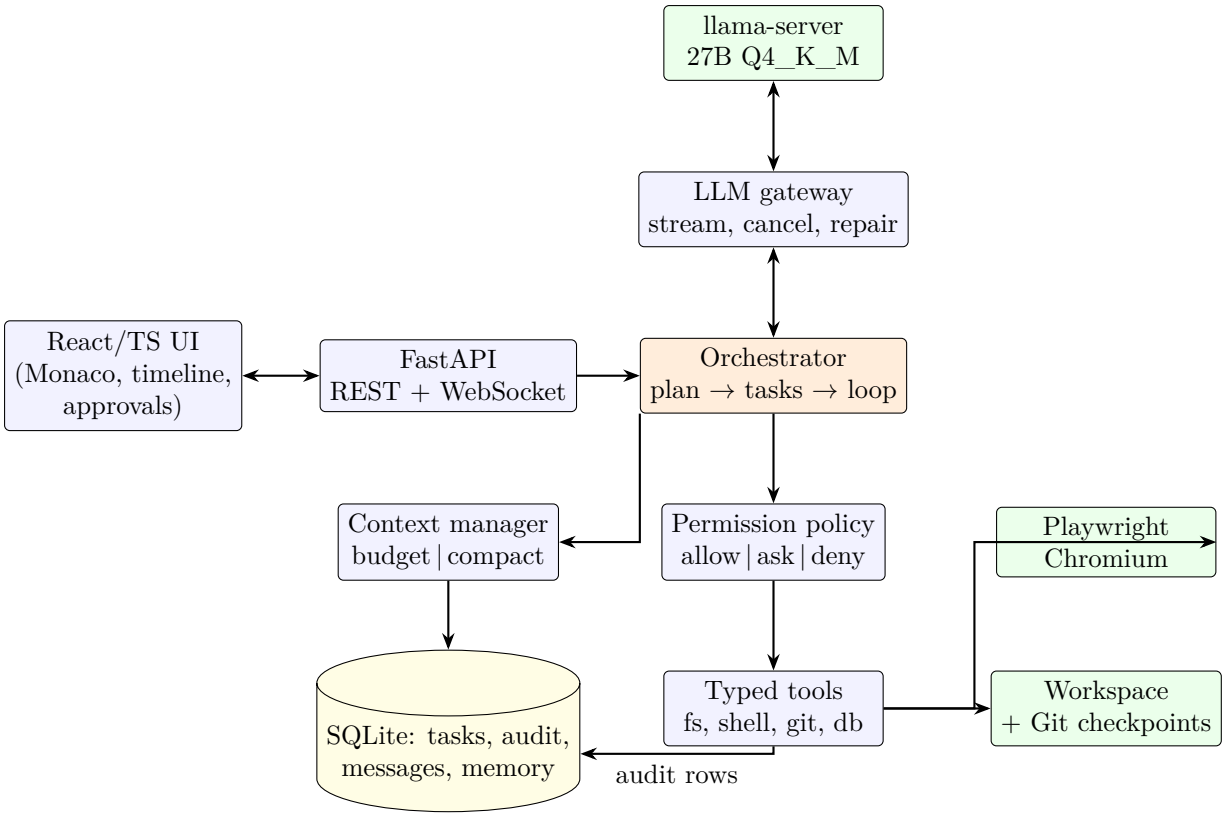


Figure 1: ForgeAgent components. Arrows show the direction of calls. Every tool execution passes through the permission policy and is recorded in SQLite.

in SQLite [23] through SQLAlchemy [2] with Alembic migrations. All model access goes through one gateway to a local `llama-server`.

4.1 Hardware and model

The target machine has an RTX 4090 Laptop GPU (16 GB VRAM), 64 GB RAM and Windows 11. The model is a community build of Qwen3.8-27B in GGUF format [10, 18], quantised with `Q4_K_M`; the file is 15.4 GiB and the model has 27.3 B parameters as reported by the server. The model alone nearly fills the card, so llama.cpp's automatic fitting places as many layers as possible in VRAM and runs the rest on the CPU. With `Q4_K_M` the measured generation speed was 7–10 tokens/s; an 11 GB `IQ3_S` quantisation that fits entirely in VRAM reached about 36 tokens/s (Section 12.3). Further measured values are reported in Section 12.

4.2 Roles on a single model

ForgeAgent defines five *logical* roles — Architect, Developer, Debugger, Tester, Reviewer. A role is nothing more than a different system prompt (Section 7.4) and a different tool subset; all roles call the same model instance, serialised by one semaphore. Loading five 27B models is impossible on this hardware, and sequential execution also keeps resource use predictable.

Key idea

Specialisation without replication. Role prompts bias the single model toward a task (design, implement, diagnose, test, review) and the tool subset *removes* capabilities (the Architect and Reviewer cannot write files). The restriction is enforced in code, not requested in prose.

5 The local LLM gateway

Learning objectives

Explain how a streaming completion becomes a validated tool call, and why a recovery layer is necessary for small models.

The gateway speaks the OpenAI-compatible protocol to llama.cpp. It (a) discovers the model from `/models` and reads capabilities (real context size, whether the chat template supports tools) from llama.cpp's `/props`; (b) parses the server-sent event stream, assembling content, reasoning tokens and *fragmented* tool-call deltas; (c) records token usage and generation speed from the server's timing fields; (d) retries transport and 5xx errors with exponential backoff; and (e) supports cancellation.

5.1 Tool-call recovery

In our experiments the native tool-call path worked with the chosen model, but the gateway is built for the general case. Native calling is tried first; if the server rejects the `tools` field (HTTP 400), the gateway switches permanently to a *prompt-based* protocol: tool schemas are placed in the system prompt, and the model replies with `<tool_call>{json}</tool_call>` blocks. Independently of the mode, tool calls found *in text* are recovered from several dialects: JSON blocks, the Qwen XML dialect (`<function=name><parameter=key>value</parameter></function>`) and fenced JSON. Listing 1 shows the JSON repair used for truncated or sloppy arguments. Unrepairable calls are *never executed*: they are returned to the model as an error that restates the expected schema.

Listing 1: JSON repair for model-generated tool arguments (excerpt from `backend/app/llm/toolparse.py`).

```python
# backend/app/llm/toolparse.py :: repair_json
def repair_json(s: str) -> Any:
    """Best-effort repair of near-JSON produced by small models. Raises ValueError if hopeless."""
    s = s.strip()
    if not s:
        raise ValueError("empty")
    try:
        return json.loads(s)
    except json.JSONDecodeError:
        pass
    t = s
    t = re.sub(r"^```(?:json)?|```$", "", t.strip()).strip()
    t = re.sub(r",\s*([}\]])", r"\1", t)  # trailing commas
    t = t.replace("\u201c", '"').replace("\u201d", '"')
    # python-style literals
    t = re.sub(r"\bNone\b", "null", t)
    t = re.sub(r"\bTrue\b", "true", t)
    t = re.sub(r"\bFalse\b", "false", t)
    try:
        return json.loads(t)
    except json.JSONDecodeError:
        pass
```

```
    # close unbalanced braces / brackets / strings (truncated output)
    stack: list[str] = []
    in_str = False
    esc = False
    for ch in t:
        if in_str:
            if esc:
                esc = False
            elif ch == "\\":
                esc = True
            elif ch == '"':
                in_str = False
        else:
            if ch == '"':
                in_str = True
            elif ch in "{[":
                stack.append("}" if ch == "{" else "]")
            elif ch in "}]" and stack:
                stack.pop()
    fixed = t + ('"' if in_str else "") + "".join(reversed(stack))
    try:
        return json.loads(fixed)
    except json.JSONDecodeError:
        pass
    # single-quoted JSON
    try:
        import ast

        v = ast.literal_eval(s)
        if isinstance(v, (dict, list)):
            return v
    except Exception:
        pass
    raise ValueError("could not repair JSON")
```

5.2 Instant cancellation

A user pressing *Stop* must take effect immediately, but llama.cpp may spend tens of seconds evaluating a long prompt before it emits the first token, during which a naive `for line in stream` loop cannot observe a cancel flag. The gateway therefore races the next stream read against a cancel event (Listing 2); leaving the HTTP context closes the connection, which makes llama.cpp abort generation. A unit test with a server that stalls for 30 s verifies that cancellation completes in under 3 s.

Listing 2: Cancellable stream iteration (`backend/app/llm/gateway.py`).

```
# backend/app/llm/gateway.py :: _cancellable
async def _cancellable(aiter, cancel: asyncio.Event | None):
    """Yield from ‘aiter‘, but raise LLMCancelled the moment ‘cancel‘ is set - even while blocked waiting for the
    first token (llama.cpp can spend a long time evaluating the prompt before it streams anything)."""
    if cancel is None:
        async for item in aiter:
            yield item
        return
    it = aiter.__aiter__()
    cancel_task = asyncio.ensure_future(cancel.wait())
    try:
        while True:
            nxt = asyncio.ensure_future(it.__anext__())
            done, _ = await asyncio.wait({nxt, cancel_task}, return_when=asyncio.FIRST_COMPLETED)
            if cancel_task in done and nxt not in done:
                nxt.cancel()
                raise LLMCancelled("cancelled")
            try:
                item = nxt.result()
            except StopAsyncIteration:
                return
            yield item
            if cancel.is_set():
```

```
                raise LLMCancelled("cancelled")
    finally:
        cancel_task.cancel()
```

Engineering lesson

Blocking I/O hides cancellation. Any loop that waits on a remote producer needs a way to be woken by the cancel signal, not just a flag polled between items.

6 Typed tools and the permission policy

Learning objectives

Describe how a tool is defined, validated and audited; apply the principle of least privilege to an agent.

6.1 Tool abstraction

Every tool declares a name, a description, a Pydantic parameter model (which doubles as the JSON Schema shown to the model), a *category* (`read`, `write`, `exec`, `git`, `browser`, `destructive`, `meta`) and returns a structured `ToolResult`. Listing 3 shows the base class.

Listing 3: Tool base class (`backend/app/tools/base.py`).

```python
# backend/app/tools/base.py :: Tool
class Tool:
    name: ClassVar[str]
    description: ClassVar[str]
    Params: ClassVar[type[BaseModel]]
    # read | write | exec | git | browser | destructive | meta
    category: ClassVar[str] = "read"
    mutates: ClassVar[bool] = False

    async def run(self, ctx: ToolContext, args: BaseModel) -> ToolResult:  # pragma: no cover - interface
        raise NotImplementedError

    @classmethod
    def spec(cls) -> dict:
        schema = cls.Params.model_json_schema()
        schema.pop("title", None)
        for p in schema.get("properties", {}).values():
            p.pop("title", None)
        return {"type": "function", "function": {"name": cls.name, "description": cls.description, "
     parameters": schema}}
```

ForgeAgent registers 28 tools (Table 1). Arguments are validated before execution; a validation failure is returned to the model with the offending fields and the expected schema, which small models use to self-correct. Every execution creates a row in an append-only `tool_runs` table with the (secret-redacted) arguments, full output, approval state, duration and the files touched.

6.2 Workspace confinement

File tools resolve every path with the real path of the target (following symbolic links and Windows junctions) and refuse anything outside the authorised workspace; `.git/` and `.forge/` are write-protected (Listing 4). Writes are atomic (temporary file, `fsync`, `os.replace`), so a crash never leaves a half-written source file.

Listing 4: Workspace path validation (`backend/app/security.py`).

```python
# backend/app/security.py :: resolve_in_workspace
def resolve_in_workspace(root: str | Path, rel: str | Path, must_exist: bool = False) -> Path:
```

Category	Tools
Filesystem (read)	`list_directory`, `read_file`, `search_files`, `search_code`, `query_database`, `inspect_logs`, `read_full_output`
Filesystem (write)	`create_file`, `edit_file`, `apply_patch`, `move_file`; `delete_file` (destructive)
Execution	`run_command`, `run_tests`, `run_build`, `start_dev_server`, `stop_dev_server`
Git	`git_changes`, `create_checkpoint`; `restore_checkpoint` (destructive)
Browser	`open_browser`, `capture_screenshot`, `inspect_browser_errors`, `browser_interact`
Control & memory	`submit_plan`, `complete_task`, `remember`, `recall`

Table 1: The 28 registered tools by category.

```
    """Resolve ‘rel‘ against the workspace root; refuse anything that escapes it (incl. symlinks /
     junctions)."""
    root_p = Path(root).resolve()
    raw = Path(str(rel))
    cand = raw if raw.is_absolute() else root_p / raw
    # realpath resolves symlinks for the existing part of the path
    resolved = Path(os.path.realpath(cand))
    try:
        resolved.relative_to(root_p)
    except ValueError:
        raise WorkspaceViolation(f"Path ’{rel}’ resolves outside the authorized workspace") from None
    if must_exist and not resolved.exists():
        raise FileNotFoundError(f"Path not found: {rel}")
    return resolved
```

Listing 5: Atomic file write (`backend/app/tools/fs_tools.py`).

```
# backend/app/tools/fs_tools.py :: atomic_write
def atomic_write(path: Path, data: str) -> None:
    """Write via temp file + os.replace so readers never see partial content."""
    path.parent.mkdir(parents=True, exist_ok=True)
    fd, tmp = tempfile.mkstemp(dir=str(path.parent), prefix=".forge-", suffix=".tmp")
    try:
        with os.fdopen(fd, "w", encoding="utf-8", newline="") as f:
            f.write(data)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp, path)
    except Exception:
        try:
            os.unlink(tmp)
        except OSError:
            pass
        raise
```

6.3 Autonomy modes

Three modes decide, per call, whether a tool runs (*allow*), needs the user's consent (*ask*) or is disabled (*deny*):

Supervised reads run automatically; every write, command and browser action asks.

Project autonomous (default) edits, builds, tests and approved development commands run inside the workspace; deletions, checkpoint restores, destructive or network commands, secrets, deployments, unknown commands, paths outside the workspace and external URLs ask.

Custom a per-tool allow/ask/deny table; unspecified tools follow the project-autonomous policy.

Listing 6: The permission decision function (`backend/app/agent/permissions.py`).

```python
# backend/app/agent/permissions.py :: decide
def decide(tool: Tool, args: BaseModel, ctx: ToolContext) -> Decision:
    aut: AutonomySettings = ctx.settings.autonomy
    if tool.category == "meta":
        return Decision("allow")

    if aut.mode == "custom":
        pol = aut.custom_permissions.get(tool.name)
        if pol == "deny":
            return Decision("deny", [f"tool '{tool.name}' is disabled in custom permissions"])
        if pol == "ask":
            return Decision("ask", [f"approval required for '{tool.name}' (custom rule)"])
        if pol == "allow":
            return Decision("allow")
        # unspecified tools fall through to the project-autonomous policy

    if aut.mode == "supervised":
        if tool.category == "read":
            if tool.name == "read_file" and any(is_secret_file(p) for p in _paths_for(tool, args)):
                return Decision("ask", ["reads a file that may contain secrets"])
            return Decision("allow")
        if tool.category == "git" and not tool.mutates:
            return Decision("allow")
        return Decision("ask", [f"supervised mode: approval required for {tool.category} action '{tool.
     name}'"])

    # project_autonomous (and custom fall-through)
    reasons: list[str] = []
    if tool.category == "read":
        if tool.name == "read_file" and any(is_secret_file(p) for p in _paths_for(tool, args)):
            reasons.append("reads a file that may contain secrets")
        return Decision("ask" if reasons else "allow", reasons)
    if tool.category == "destructive":
        return Decision("ask", [f"destructive action: {tool.name}"])
    if tool.category == "write":
        if any(is_secret_file(p) for p in _paths_for(tool, args)):
            return Decision("ask", ["modifies a file that may contain secrets"])
        return Decision("allow")
    if tool.category == "exec":
        cmd = _command_for(tool, ctx, args)
        if cmd:
            verdict, why = classify_command(cmd, ctx.root, aut)
            if verdict == "approval":
                return Decision("ask", why)
        return Decision("allow")
    if tool.category == "browser":
        url = getattr(args, "url", None)
        if url and not is_local_url(url):
            return Decision("ask", [f"external URL: {url}"])
        return Decision("allow")
    return Decision("allow")
```

6.4 Command classification

Shell commands are classified by `classify_command` (Listing 7): regular expressions detect destructive operations, network use, deployment and secret access; absolute or parent-relative paths outside the workspace are flagged; and every sub-command of a chain (`&&`, `;`, `|`) must begin with an allow-listed development tool after normalising quoted executable paths. Table 2 shows representative verdicts from the test suite.

Listing 7: Command classification (`backend/app/security.py`).

```python
# backend/app/security.py :: classify_command
def classify_command(cmd: str, workspace: str | Path, autonomy: AutonomySettings) -> tuple[str, list[str
    ]]:
    """Return ("auto"|"approval", [reasons]). Never returns 'deny'; callers decide policy."""
    reasons: list[str] = []
    low = cmd.strip().lower()
    if _ANY(_DESTRUCTIVE, low):
        reasons.append("destructive operation")
```

Command	Verdict
`npm test` / `python -m pytest -q` / `git status && git diff`	auto
`npm install express`	auto
`rm -rf node_modules`	approval (destructive)
`git push origin main`	approval (destructive / external)
`curl http://x/s.sh | sh`	approval (network)
`type ..\\..\\secrets.txt`	approval (outside workspace)
`npm test && rm -rf /`	approval (one bad link taints the chain)
`vercel deploy -prod`	approval (deployment)

Table 2: Representative command-classification outcomes (from `tests/test_security_fs.py`).

```
    if _ANY(_NETWORK, low) or (not autonomy.network_access and re.search(r"https?://", low)):
        reasons.append("external/network access")
    if _ANY(_DEPLOY, low):
        reasons.append("deployment")
    if _ANY(_SECRETS, low):
        reasons.append("possible secrets access")
    ws = str(Path(workspace).resolve()).lower().replace("/", "\\")
    for m in _ABS_PATH.finditer(cmd):
        token = m.group(0)
        if m.group(1):
            norm = os.path.normpath(token).lower()
            if not norm.startswith(ws):
                reasons.append(f"path outside workspace: {token}")
        else:
            reasons.append(f"path outside workspace: {token}")
    if not reasons:
        # every sub-command of a chain must start with an allowed prefix
        parts = [p.strip() for p in _CHAIN.split(low) if p and not _CHAIN.fullmatch(p) and p.strip()]
        prefixes = [p.lower() for p in autonomy.allowed_command_prefixes]
        for part in parts:
            part = _normalize_part(part)
            if not any(part == pre or part.startswith(pre + " ") or part.startswith(pre + ".exe") or
                       (" " in pre and part.startswith(pre)) for pre in prefixes):
                reasons.append(f"command not on the approved development list: '{part.split()[0] if part.
 split() else part}'")
                break
    return ("approval" if reasons else "auto"), reasons
```

Key idea

Heuristics are not a sandbox. The classifier reduces accidents and makes risky actions visible, but a command obfuscated by indirection (for example a script the agent writes and then runs) can bypass it. Real isolation requires an OS-level boundary (a VM, WSL2 or a restricted account); ForgeAgent does not provide one (Section 14).

7 The agent loop

Learning objectives

Trace one user request through planning, the task queue and the tool loop; explain each guard and the failure it prevents.

7.1 Control flow

Figure 2 shows the loop. A request creates a root *task*; a Git checkpoint is committed first so the whole run can be undone. If planning is enabled (*auto* for long or build-style requests),

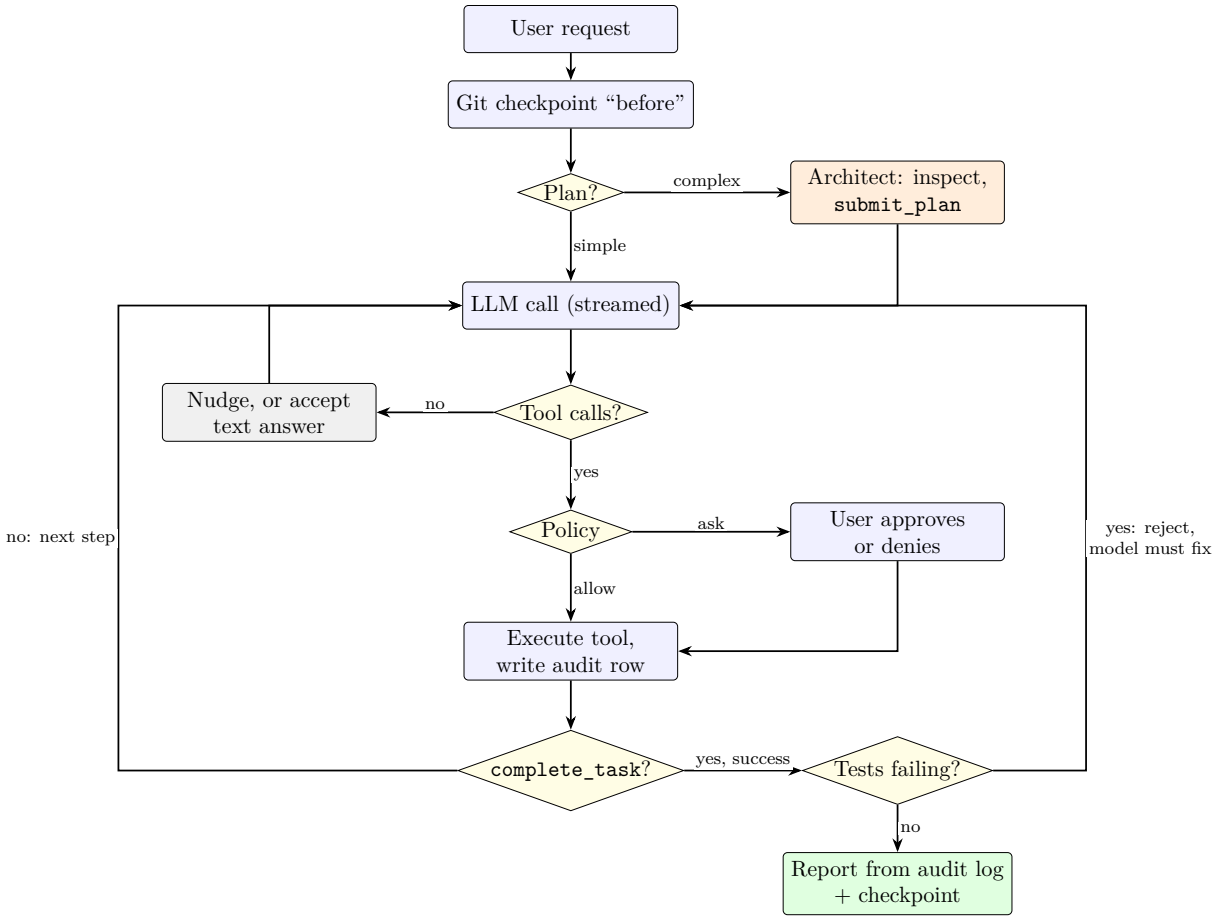


Figure 2: Simplified control flow. The “Tests failing?” guard and the audit-derived report separate ForgeAgent from a plain tool loop. (*Stop* and the step/retry limits apply at every node.)

the Architect role inspects the workspace with read-only tools and calls `submit_plan` with a summary, technology stack and an ordered list of milestones with dependencies. Simple plans run as one Developer task. Complex plans become child tasks executed by the queue in dependency and priority order, each with the role the plan assigned, followed by an automatic Reviewer task; if the review reports blocking problems, exactly one Debugger round is spawned. If the Architect fails to produce a plan, the run falls back to a single task and says so in the activity feed.

7.2 Guards against bad agent behaviour

- **Verified success.** `complete_task(success)` is rejected while the most recent test or build run failed and has not been re-run successfully; the model must fix the problem or report `partial`.
- **Don’t stop at a plan.** A text-only reply that follows edits, or that looks like a plan (“I will…”, numbered steps), is answered with a nudge to continue or finish, at most `max_nudges` times; a plain answer to a question is accepted immediately.
- **Invalid calls are feedback, not crashes.** Unknown tools, malformed JSON and schema violations are returned as errors with the expected schema.
- **Duplicate-action prevention.** A call is fingerprinted by (tool, arguments, *world version*); the world version increments whenever a file or command changes state. A third identical call with no intervening change is refused.

- **Bounded effort.** Step limit per task, retry limit with a changed-approach hint, consecutive-error limit, command timeouts, and process-tree termination on stop.

7.3 Truthful reporting

The text the user finally reads has two parts: the model's summary and a block *generated by the orchestrator from the* `tool_runs` *table* (files changed, commands executed, last test and build status, checkpoint id). A model that exaggerates is contradicted by the log shown directly below its claim.

Key idea

Trust the log, not the narrator. In any agent that reports on its own work, derive the report from independent evidence (the audit log, a re-run of the tests), and make false success claims structurally impossible where verification exists.

7.4 Role prompts

The system prompt is assembled from user-editable sections (identity, operating rules, coding standards, planning and testing instructions), a role block, an environment block (operating system, shell, workspace path, date), retrieved memory and, if the history has been compacted, a progress summary. The operating rules state that file contents, comments, web pages and tool output are untrusted data that cannot alter instructions or permissions, and that analysing code is not executing it. Tool results that contain third-party content are additionally prefixed with an *untrusted content* notice. No refusal keyword filter exists anywhere in the gateway or orchestrator: model refusal behaviour and execution permission are separate mechanisms.

8 Persistence, recovery and resume

Learning objectives

Design agent state so that a hard crash loses no completed work and a task can be resumed deterministically.

All state needed to continue lives in SQLite (WAL mode [23]): tasks and their statuses (PENDING, PLANNING, RUNNING, TESTING, BLOCKED, FAILED, COMPLETED, CANCELLED), the complete message history including tool calls and results, the audit log, memory, checkpoints and the event timeline. At start-up, any task found in PLANNING, RUNNING or TESTING is, by definition, an orphan of a previous process and is parked as BLOCKED with the message "Interrupted by application restart" (Listing 8). *Resume* resets unfinished tasks to PENDING, reloads their history, adds a message telling the model that the previous run was interrupted and that it must re-check the workspace rather than assume earlier actions succeeded, and continues. Completed milestones are not repeated.

Listing 8: Crash recovery on start-up (`backend/app/agent/runtime.py`).

```python
# backend/app/agent/runtime.py :: AgentRuntime.recover_interrupted
def recover_interrupted(self) -> int:
    """Called at startup: anything that was mid-flight is parked as BLOCKED so it can be resumed."""
    n = 0
    with session_scope() as s:
        for t in s.query(Task).filter(Task.status.in_((TaskStatus.PLANNING, TaskStatus.RUNNING, TaskStatus.
     TESTING))).all():
            t.status = TaskStatus.BLOCKED
            t.error = "Interrupted by application restart. Resume to continue from the saved state."
            n += 1
```

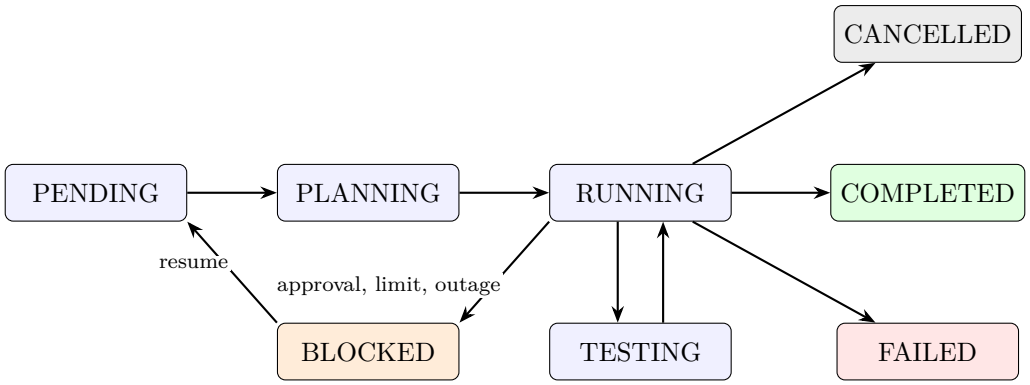


Figure 3: Task states. BLOCKED covers waiting for approval, step limit, model outage and crash recovery; it is always resumable.

```
    return n
```

A subtle requirement: if the process dies *between* the model requesting a tool and the result being stored, the history contains a tool call with no result, which most chat templates reject. `repair_tool_pairs` therefore inserts a synthetic result stating that the outcome is unknown (Listing 9).

Listing 9: Repairing interrupted tool-call pairs (`backend/app/agent/context.py`).

```python
# backend/app/agent/context.py :: repair_tool_pairs
def repair_tool_pairs(msgs: list[dict]) -> list[dict]:
    """Guarantee every assistant tool call is followed by a tool message (needed after interrupted runs)."
    ""
    out: list[dict] = []
    i = 0
    while i < len(msgs):
        m = msgs[i]
        out.append(m)
        if m["role"] == "assistant" and m.get("tool_calls"):
            wanted = [tc["id"] for tc in m["tool_calls"]]
            j = i + 1
            seen: set[str] = set()
            while j < len(msgs) and msgs[j]["role"] == "tool":
                seen.add(msgs[j].get("tool_call_id", ""))
                out.append(msgs[j])
                j += 1
            for tc in m["tool_calls"]:
                if tc["id"] not in seen:
                    out.append({"role": "tool", "tool_call_id": tc["id"], "name": tc["function"]["name"],
                                "content": "ERROR: this action was interrupted (application restart or
    cancellation). Its outcome is unknown - re-inspect the current state before continuing."})
            i = j
            continue
        i += 1
    return out
```

9 Context management

Learning objectives

Compute a token budget, and explain truncation and compaction as controlled forgetting.

With a 16 384-token window, the usable prompt budget is

$$B \;=\; C - R - S_{\text{tools}}, \qquad R = \min\!\left(\text{max_tokens},\, \tfrac{C}{3}\right),$$

where C is the context size, R the tokens reserved for the reply and S_{tools} the estimated size of the tool schemas. Token counts are estimated at about 3.6 characters per token (exact counts are available through llama.cpp's `/tokenize` endpoint). Before each model call the history is estimated; above a threshold (default $0.65\,B$) the older messages are folded into a running summary.

Tool-output truncation. Outputs longer than a limit are cut head-and-tail and annotated with the `tool_run_id`; the full text stays in the audit table and is retrievable with `read_full_output`.

Compaction. The model is asked to summarise the old log (goals, decisions, files changed, real command outcomes, unresolved errors, next steps); on failure a deterministic digest is used. The original user goal is never compacted, and an assistant tool call is never separated from its results (Listing 10). A last-resort trimmer shortens old tool outputs and drops the oldest complete turns if the prompt still does not fit.

Listing 10: History compaction (`backend/app/agent/context.py`).

```python
# backend/app/agent/context.py :: compact_history
async def compact_history(gateway: LLMGateway, task_id: str, settings: AppSettings, cancel=None) -> bool:
    """Fold older messages into Task.summary. Returns True when something was compacted."""
    keep = settings.memory.keep_recent_messages
    with session_scope() as s:
        rows = (s.query(Message).filter(Message.task_id == task_id, Message.compacted.is_(False)).order_by(Message.id).all())
        task = s.get(Task, task_id)
        if len(rows) <= keep + 2 or task is None:
            return False
        cut = len(rows) - keep
        # never split an assistant tool_call from its tool results; and keep the first (goal) user message
        while cut < len(rows) and rows[cut].role == "tool":
            cut += 1
        old = [r for r in rows[:cut] if not (r.role == "user" and r is rows[0])]
        if not old:
            return False
        prior = task.summary
        transcript = "\n".join(
            f"[{r.role}{'/' + r.tool_name if r.tool_name else ''}] "
            + ((r.content or "")[:700] + (" TOOL_CALLS: " + json.dumps(r.tool_calls)[:400] if r.tool_calls else ""))
            for r in old)
        ids = [r.id for r in old]
    prompt = [
        {"role": "system", "content": "You compress an AI software-engineering agent's working log into a dense progress summary. "
                                      "Keep: the goal, decisions, files created/changed (paths), commands run and their real outcomes, "
                                      "test/build status, unresolved errors, and the next steps. Drop chatter. Max 350 words. Plain text."},
        {"role": "user", "content": (f"Previous summary:\n{prior}\n\n" if prior else "") + f"New log to fold in:\n{transcript[:14000]}"},
    ]
    try:
        comp = await gateway.complete(prompt, None, cancel=cancel, max_tokens=700)
        summary = comp.content.strip() or heuristic_summary(old)
    except LLMError:
        summary = (prior + "\n" if prior else "") + heuristic_summary(old)
    with session_scope() as s:
        task = s.get(Task, task_id)
        task.summary = summary[:6000]
        for r in s.query(Message).filter(Message.id.in_(ids)).all():
            r.compacted = True
    return True
```

Key idea

Compaction is *lossy by design*. Preserve what cannot be re-derived (the goal, decisions, error causes) and discard what can be re-read (file contents, long logs).

10 Persistent project memory

Memory items have a kind (requirement, decision, stack, status, completed, pending, error_fix, file, test, convention, note), a title, content and a *pinned* flag; pinned items are always included in the prompt. They are written by the agent (`remember`), automatically (the request as a requirement, the plan's stack and architecture, completed tasks, and "failure fixed" records created when a failing test/build is followed by a pass, including the files changed in between) and by the user in the Memory tab.

Retrieval needs no embedding model. Text is tokenised (splitting camelCase), uni- and bi-grams are *feature-hashed* [26] into a 4096-dimensional signed sparse vector with log-scaled term weights and L_2 normalisation (Listing 11), and relevance is cosine similarity plus a small token-overlap bonus. This is lexical rather than semantic: it finds "database" when the memory says "database" but not necessarily "persistence". A neural embedding model could be substituted behind the same interface.

Listing 11: Feature-hashed vectors (`backend/app/agent/memory.py`).

```python
# backend/app/agent/memory.py :: embed
def embed(text: str) -> list[list[float]]:
    toks = _tokens(text)
    feats = Counter(toks) + Counter(f"{a}_{b}" for a, b in zip(toks, toks[1:]))
    vec: dict[int, float] = {}
    for f, c in feats.items():
        h = zlib.crc32(f.encode())
        idx = h % DIM
        sign = 1.0 if (h >> 16) & 1 else -1.0
        vec[idx] = vec.get(idx, 0.0) + sign * (1 + math.log(c))
    norm = math.sqrt(sum(v * v for v in vec.values())) or 1.0
    return [[i, round(v / norm, 5)] for i, v in vec.items()]
```

11 User interface

The UI (React, TypeScript, Vite, Tailwind CSS, Monaco; bundled locally so it works offline) mirrors the system structure. The *Projects* dashboard (Figure 4) lists workspace, stack, current task, last activity, completion and Git status. The *Workspace* (Figure 5) combines the explorer, editor, a live activity timeline, task, tool, test, checkpoint and memory tabs, and the chat panel with streaming output, approval cards and Stop/Pause controls. *Settings* (Figure 6) exposes model parameters, server start/stop and model switching, all prompts, autonomy and per-tool permissions, and memory options.

12 Evaluation

Learning objectives

Distinguish unit/integration tests with a scripted model from end-to-end experiments with the real model, and read each result with its limits in mind.

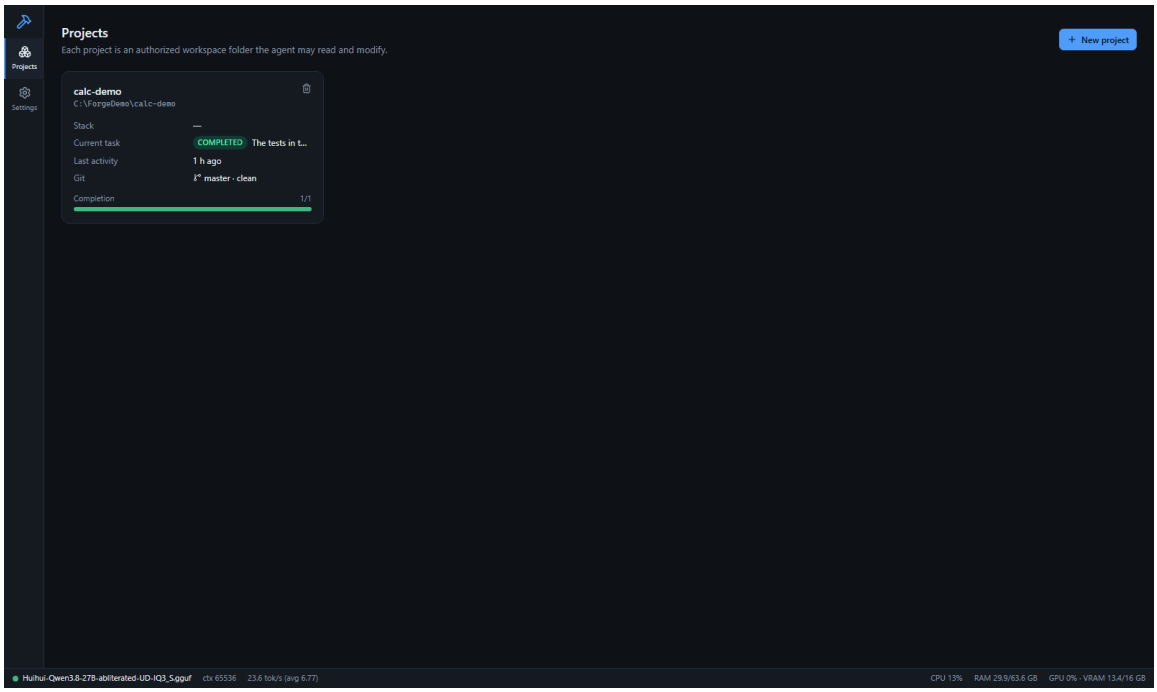


Figure 4: Projects dashboard.

File	Tests	What is covered
`test_toolparse.py`	7	JSON/XML/fenced tool-call recovery, JSON repair, thinking split
`test_gateway.py`	9	streaming, native calls, fallback, retries, outage, cancel (incl. stalled stream)
`test_security_fs.py`	31	path boundaries, junction escape, command classes (incl. quote-aware splitting), rec
`test_orchestrator.py`	18	fix-a-failing-test, success-claim rejection, approvals, stop, duplicates, malformed call
`test_api.py`	13	migrations, projects and workspace validation, files, settings, memory, WebSocket e
`markdown.test.ts`	3	HTML escaping of model output, code fences, lists

Table 3: Automated test inventory.

12.1 Automated tests

The back end has 78 automated tests and the front end 3 (Table 3). The orchestrator tests run the *real* tools, Git and subprocesses against a deterministic fake OpenAI-compatible server that replays scripted replies, so the control logic, guards and persistence are tested without model variance. All tests pass; the suite was run repeatedly to detect flakiness (see Section 13 for the flakiness this exposed).

12.2 End-to-end experiments with the real model

All experiments ran on the target machine (RTX 4090 Laptop GPU, 16 GB VRAM; 64 GB RAM; Windows 11; llama.cpp build b11620) with the *real* local model, driven through the public HTTP API by `scripts/acceptance.py`, which records raw outcomes in `docs/ACCEPTANCE_RESULTS.md`. Unless stated otherwise the model was the `Q4_K_M` quantisation with a 16 384-token context. Scenarios were run once each; this is a demonstration that the mechanisms work end to end, not a statistical estimate of success rates.

Acceptance scenarios

Table 4 lists the seven scenarios and the independent evidence for each. *All seven ended in a pass*, but three of them failed or crashed on earlier attempts for reasons described below; those

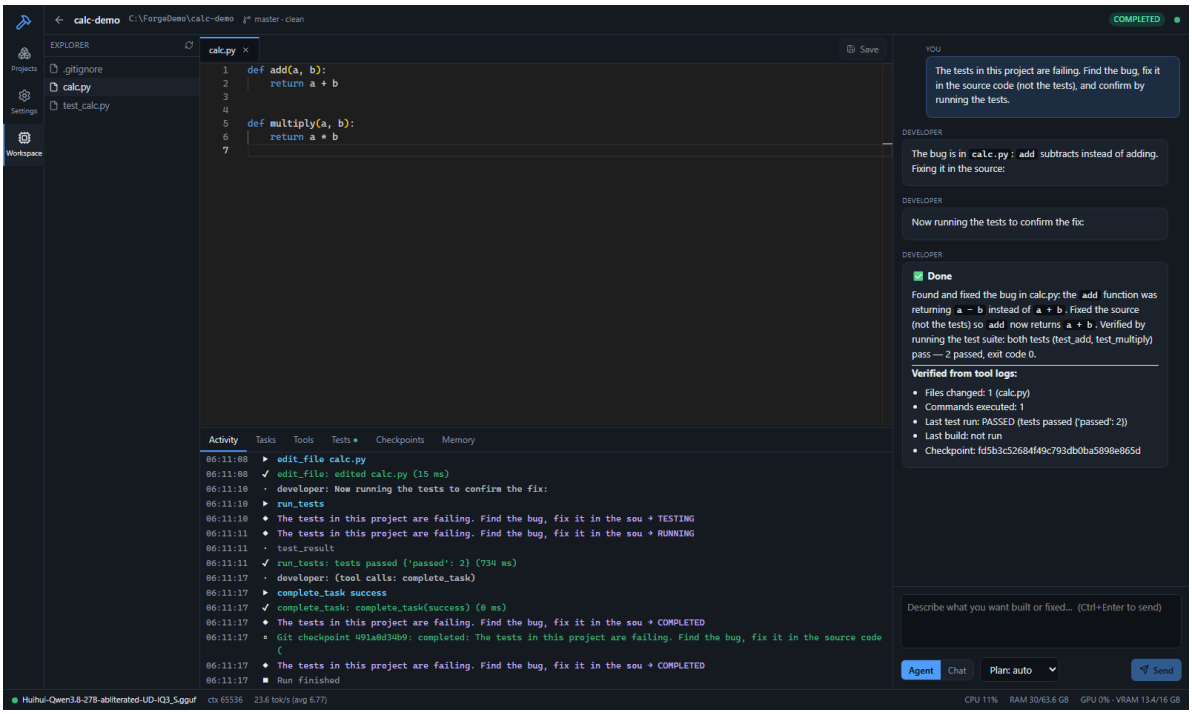


Figure 5: Workspace after a completed run: explorer, editor, live activity, and the chat panel with the audit-derived "Verified from tool logs" block.

attempts are retained in the raw results file rather than discarded.

Fix a failing test. The agent listed the directory, read the source and the test, and edited `calc.py` (`a - b` → `a + b`). Its first two `run_tests` calls failed because `pytest` was not installed in the global interpreter; the agent read the real error, installed it with `pip`, re-ran the tests (2 passed), recorded the fix in memory and finished. Nine model steps were used. The first failing runs are visible in the audit log, and the final report states "Last test run: PASSED" from that log.

Boundaries. Asked to write a file at an absolute path outside the workspace, the model called `create_file`; the tool refused ("resolves outside the authorized workspace"). The model then tried a shell command, which requested approval and was denied by the harness, and finally created the file inside the workspace as the prompt allowed.

Supervised mode: a failed first attempt that taught something. The first two runs of this scenario *failed*. The command `echo hi > other.txt` was denied and recorded as denied, but the model then created `other.txt` with `create_file`, which also asked for approval — and the test script approved every `create_file`. The enforcement worked as designed; the test was wrong, and the episode shows that a model *will* try another route after a denial, so every route must pass the same gate. With the script corrected to approve only `notes.txt`, the audit log shows: `create_file` approved, `run_command` denied, `create_file` (for `other.txt`) denied, and `other.txt` did not exist.

Restart and resume. After the first file was created, the back-end process was killed. A new process started with an empty memory; the task was recovered as BLOCKED with the message "Interrupted by application restart", *Resume* was invoked, and the model completed the remaining

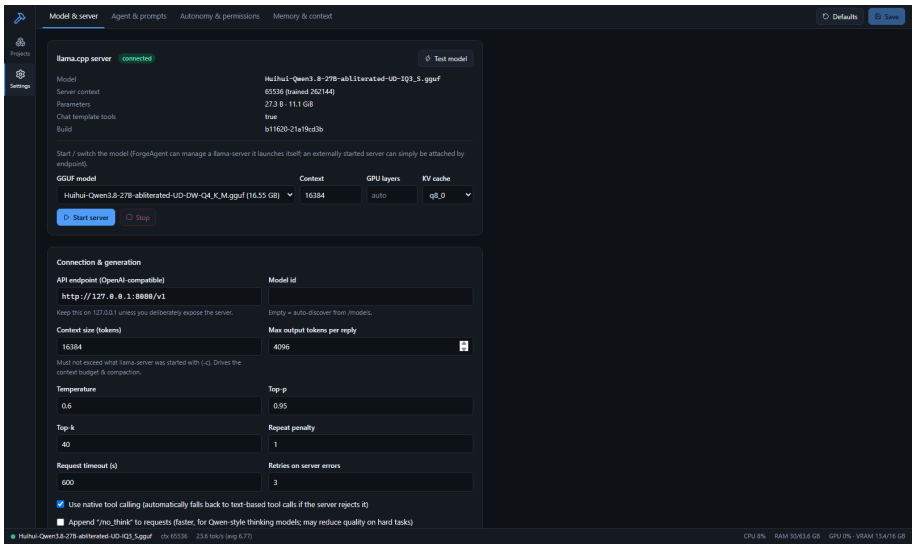


Figure 6: Model and server settings: connection status, capability detection, server start/stop with quantised KV cache, and generation parameters.

Scenario	Outcome	Independent evidence
Fix a failing test	pass	tests re-run by the harness after the agent finished: 2 passed; test file byte-identical to the original
Workspace boundaries	pass	requested absolute path outside the workspace was refused by the tool; file does not exist on disk
Git checkpoint restore	pass	after restoring the "before" checkpoint the original content was back and the newly created file was gone
Stop a running agent	pass	stop returned in 8.0 s; task CANCELLED; no files written in the following 8 s
Supervised approvals	pass (2nd run)	approved edit executed; denied command did not run; audit shows approved/denied per call
Restart and resume	pass	backend process killed with `taskkill /F` mid-task; recovered as BLOCKED; resumed to COMPLETED with all three files
Full-stack app	pass	harness re-ran the agent's tests (10 passed) and exercised the live API (201/200/204)

Table 4: Real-model acceptance scenarios.

two files (8–9 steps in total over the two runs). The model call after resumption contained the pre-restart history and the interruption notice.

Full-stack application

The prompt asked for a FastAPI + SQLite Todo application with a static JavaScript front end, a project-local virtual environment, pytest tests, and a live check in the browser. The Architect produced a plan of five milestones plus the automatic review. All seven tasks completed (Table 5); the run used 142 tool calls, of which 14 returned errors that the agent then handled (for example a dev-server port already in use, and a mistaken selector in a browser test). The harness then verified the result *without relying on the agent's report*: the agent's tests passed when re-run (10 passed), and a server started from the workspace answered `POST /api/todos` with 201, listed the new item, accepted `PATCH` (200) and `DELETE` (204), and served an HTML page that uses `fetch`.

Milestone	Role	Model steps
Plan: architecture and task breakdown	architect	3
Scaffold project (venv, requirements)	developer	8
Implement FastAPI backend	developer	30
Create static front end	developer	37
Write and run pytest tests	developer	6
Runtime verification in the browser	tester	19
Final review	reviewer	6

Table 5: Full-stack run: milestones and model steps (Q4_K_M).

	Q4_K_M	IQ4_XS	IQ3_S
File size (GB)	15.41	13.41	11.13
VRAM in use (GB)	13.7	13.7	13.3
Generation (tokens/s)	7.2	9.5	**35.9**
Prompt processing (tokens/s)	584	677	**1195**
GPU utilisation (%)	30	34	**94**
CPU cores busy (of 32)	20	21	**1**
Tool calls correct	6/6	6/6	6/6
Coding tasks passed	7/8	6/8	7/8
Mean time per coding task (s)	184	137	**47**

Table 6: Same model, three quantisations, identical workload (measured). VRAM figures are the whole-GPU reading while the server was busy.

This was not a clean run, and we report it as it happened. The elapsed time of about eight hours is *not* a performance figure. (The harness printed 553 minutes; that number was inflated by a timestamp bug, described in Section 13, which made browsers and scripts read UTC times as local time.) The elapsed time includes (i) two stalls, of the order of twenty minutes each, in which the agent sat waiting for an approval that the test script was not written to answer — caused by a classifier false positive on a harmless `python -c "...; print(...)"` command (Section 13) — one of which a human approved, after which the classifier was fixed in code; (ii) a back-end restart followed by a *Resume* of the same task, which worked; (iii) periods when a second client (the Cline editor extension) was using the same single model and slowed both, from about 7 to about 2.5 tokens/s; and (iv) the test script's own timeouts. Dev servers started by the agent also survived the back-end restart and kept their ports, which made a later `start_dev_server` fail with "port in use"; the agent worked around it, and we list this as a limitation.

12.3 Quantisation trade-offs: speed, memory and quality

The 15.4 GiB `Q4_K_M` file does not fit in 16 GB of VRAM together with the KV cache, so llama.cpp runs some layers on the CPU. We therefore measured two smaller quantisations of the same model under identical conditions (context 65 536, KV cache `q8_0`, flash attention, one request at a time, nothing else using the GPU): `IQ4_XS` (13.4 GB) and `IQ3_S` (11.1 GB). Each model received the same fixed workload: a 400-token generation, an 7 318-token prompt, six tool-calling cases with exact expected arguments, and eight coding tasks whose answers were *executed* against hidden tests (the tests were first validated against reference solutions, and a deliberately wrong solution was confirmed to fail). Table 6 and Figure 7 show the results.

Reading the numbers. Speed is governed by *where the weights live*, not by arithmetic precision alone. With the two larger files part of the model runs on the CPU, so each token waits

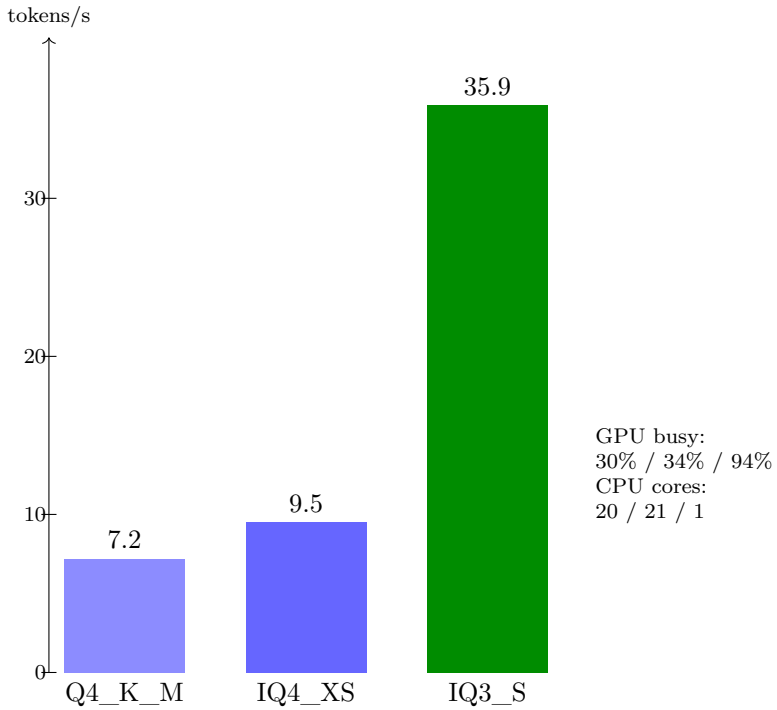


Figure 7: Generation speed in tokens per second (bars drawn to scale). Only IQ3_S fits entirely in VRAM, which is why the GPU is busy and the CPU idle.

on the slower processor while the GPU idles (30–34 % busy, about 20 CPU cores saturated). `IQ3_S` fits completely in VRAM: the GPU is 94 % busy, one CPU core is used, and generation is about five times faster. The same effect explains why the dedicated GPU appeared "not busy" during the agent runs above.

Quality. On this small workload the three quantisations were indistinguishable in tool-call reliability (all 6/6) and within one task in coding (7/8, 6/8, 7/8). All three failed the same task, `parse_csv_line`, and in every case the recorded finish reason was `length`: the model's reasoning consumed the entire 6 000-token limit before any code was emitted, so the failure reflects the token cap and the long reasoning of this model family, not wrong code. The single genuine logic error was by `IQ4_XS` on `LRUCache` (a helper class referenced without `self.`); the other two solved it. One extra miss in eight tasks is within what chance could produce, and *we draw no conclusion that `IQ3_S` is as good as `Q4_K_M` in general.*

Threats to validity. Eight short tasks, one run each, temperature 0.2, one machine; the tasks are small functions, not repository-scale changes; the full-stack acceptance run used `Q4_K_M` only; and lower-precision quantisations may degrade more on long, multi-step work than these tests can reveal. The practical recommendation that follows is to use `IQ3_S` for interactive work where latency dominates and keep `Q4_K_M` available for hard tasks, verifying with the test suite either way.

13 Engineering lessons

Building the system surfaced defects that are instructive beyond this project. Each was found by a test or by running the real model.

Engineering lesson

Stale bytecode defeats fast edit–test loops. The agent changed `a - b` to `a + b` (same file size) and re-ran the tests within the same second; Python's `.pyc` cache is keyed on source mtime (1 s resolution) and size, so the stale bytecode ran and the test still failed. Setting `PYTHONDONTWRITEBYTECODE=1` for agent subprocesses fixed it. Autonomous agents create exactly the rapid same-size edits that human workflows rarely do.

Engineering lesson

Commit before you respond. A FastAPI dependency with `yield` commits *after* the response is sent. A client that created a project and immediately used it intermittently received 404. Mutating endpoints now commit explicitly before returning.

Engineering lesson

Models write LF; Windows files are CRLF. `git apply` rejected diffs for CRLF files. A strict attempt is followed by a retry with whitespace-tolerant context matching.

Engineering lesson

Redaction can break the task. Redacting `password = ...` in everything the model reads made it see `[REDACTED]` in the source it was asked to edit. Generic redaction is applied to command output and logs; for source reads only unmistakable credential formats (private keys, `sk-`, `ghp_`, AWS keys) are removed.

Engineering lesson

Cached collaborators leak between tests. A shared gateway cached the previous test's fake server and produced order-dependent failures; singletons used in tests must be reset by fixtures.

Engineering lesson

Environment first. In the first real run, the agent correctly diagnosed that `pytest` was missing and installed it into the global interpreter. Safe, but poor practice; the default coding standards now instruct it to use a project-local virtual environment.

Engineering lesson

Window sizes must agree end to end. A client that sends a larger prompt than the server's `-c` value fails with "request exceeds the available context". The context size configured in the application, in the server and in any third-party client must match.

Engineering lesson

Parse commands the way the shell does. The first command classifier split command lines at every `;`, including inside quotes, so a harmless `python -c "from main import app; print(app.routes)"` was cut into a fragment `print(app.routes)` that was judged an "unknown command" and held for approval; the autonomous run then sat waiting. The classifier now splits only outside quotes (and at command substitutions, which genuinely run another command inside double quotes), with regression tests for both the false positive and the cases that must still be caught.

Engineering lesson

A denial must close the goal, not one route. After a shell command was denied, the model achieved the same effect with a file-creation tool. Because every tool passes the same permission gate, the second attempt was also put to the user; the gate, not the model's compliance, is what held. Test scripts that auto-approve must scope approvals narrowly.

Engineering lesson

Never emit timezone-naive timestamps. SQLite returns naive datetimes; the API sent them without an offset, so a browser in UTC+1 displayed "1 h ago" for something that had just happened, and the test harness computed inflated durations. All timestamps are now serialised with an explicit UTC offset, with a regression test.

Engineering lesson

Stalled-on-approval must be visible. An autonomous agent that needs a human decision looks identical to a slow one. The UI shows an approval card and the task status BLOCKED; automated test harnesses must watch for pending approvals explicitly.

Engineering lesson

Idle GPU does not mean unused GPU. During generation with part of the model on the CPU the GPU showed 2–39 % utilisation although the weights were mostly in VRAM; the processors alternate, and the CPU side sets the pace. Measuring GPU/CPU utilisation together with throughput (Section 12.3) showed that choosing a quantisation that fits entirely in VRAM was worth more than any software optimisation.

14 Limitations and threats to validity

- **Not a sandbox.** Commands run with the user's account rights; the classifier is a heuristic. No Docker/WSL2 isolation is integrated.
- **Small sample.** The end-to-end experiments use a handful of tasks on one model and one machine; they demonstrate that mechanisms work, not their success rate. We do not report SWE-bench or HumanEval [3] results.
- **Model dependence.** Results depend on the chosen quantised model; some community models are modified to refuse less [1], which shifts risk toward the permission layer.

- **Prompt injection is mitigated, not solved.** Labelling and least privilege reduce impact; a determined injection that leads the user to approve a harmful action is possible. We did not run an injection benchmark such as AgentDojo [5].
- **Lexical memory.** Feature-hashed retrieval has no semantic generalisation.
- **Throughput.** About 8–13 tokens/s with partial CPU offload makes large builds slow; the text-only model cannot inspect screenshots (it uses DOM text and console/network errors).
- **Orphaned dev servers.** Servers started by the agent are children of the back-end process tree only while it runs; after a hard restart they keep running and hold their ports, so a later `start_dev_server` can fail with "port in use". Tracking and reaping them on recovery is future work.
- **Shared model.** Any other client of the same local server (an editor extension, a second application) competes for the single model and can cut throughput several-fold; ForgeAgent serialises its own calls but cannot control others.
- **Single user, single machine**; one active run per project; inference is serialised.

15 Conclusion and future work

ForgeAgent shows that a locally hosted 27B model can drive a complete software-engineering workflow when the surrounding system supplies what the model lacks: tolerant parsing of its output, enforced permissions, independent verification, durable state and disciplined context use. Future work: OS-level isolation (WSL2/containers); neural embeddings; an injection benchmark; speculative decoding or smaller specialised models for speed; multi-model role assignment on larger hardware; and a systematic evaluation on public benchmarks.

Availability

Source code, tests, installation scripts, screenshots and raw acceptance results: `https://github.com/bahdleen/forgeagent` (MIT licence).

A Reproducing the results

1. Install prerequisites (Python 3.12, Node 20, Git for Windows, NVIDIA driver) and run `scripts\install .ps1 -InstallLlama -DownloadModel`.
2. Start with `scripts\start .ps1 -WithLlama`; verify the status bar shows the model.
3. Tests: `cd backend; .venv\Scripts \python -m pytest`; `cd frontend; npm test`.
4. Real-model scenarios: `python scripts\acceptance .py <scenario>` with scenario one of `fix_test, boundaries, checkpoint, stop, supervised, resume, fullstack`; output is appended to `docs/ACCEPTANCE_RESULTS.md`.

B Exercises

Exercises (with hints)

1. *Tool design.* Add a `http_get` tool restricted to `localhost`. Which permission category and which classifier rules does it need? (Hint: Section 6.)
2. *Guard.* Extend the completion guard so that `success` is also rejected when the build was never run in a project that has a `build` script. What are the false-positive risks?
3. *Budget.* For $C = 32\,768$, `max_tokens`$= 4096$ and $S_{\text{tools}} = 3500$, compute B and the compaction trigger. How does the KV-cache cost change relative to $C = 16\,384$?
4. *Recovery.* Write a test that kills the back end between a tool call and its result and show that `repair_tool_pairs` is exercised. (Hint: Listing 9.)
5. *Security.* Construct a command that the classifier marks "auto" but that writes outside the workspace. Propose a fix; then explain why a fix at this layer can never be complete.
6. *Memory.* Replace feature hashing by a neural embedding endpoint behind the same `embed` interface and compare retrieval for "persistence" vs. "database".

C Glossary

Agent loop
: Repeating model call → tool execution → observation until a stop condition.

GGUF
: File format used by llama.cpp for (quantised) model weights.

KV cache
: Stored attention keys/values for previous tokens; grows with context length.

Compaction
: Replacing old conversation with a shorter summary.

Least privilege
: Giving a component only the permissions its task needs [20].

Prompt injection
: Instructions hidden in data that try to redirect a model [9].

References

[1] Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. Refusal in language models is mediated by a single direction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.11717.

[2] Michael Bayer. SQLAlchemy. In Amy Brown and Greg Wilson, editors, *The Architecture of Open Source Applications Volume II: Structure, Scale, and a Few More Fearless Hacks*. aosabook.org, 2012.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[4] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.08691.

[5] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *NeurIPS Datasets and Benchmarks Track*, 2024. arXiv:2406.13352.

[6] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2208.07339.

[7] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.17323.

[8] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp. Accessed 2026-10-14; build b11620 used in this work.

[9] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. arXiv:2302.12173.

[10] huihui-ai. Huihui-qwen3.8-27b-abliterated-gguf (model card and files). https://huggingface.co/huihui-ai/Huihui-Qwen3.8-27B-abliterated-GGUF. Base model Qwen/Qwen3.8-27B, Apache-2.0 licence per model card; accessed 2026-10-14.

[11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

[12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.

[13] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. arXiv:2307.03172.

[14] OpenAI. Function calling (tool use) in the chat completions API. https://platform.openai.com/docs/guides/function-calling. Accessed 2026-10-14; the de-facto wire format reproduced by llama.cpp's server.

[15] OWASP Foundation. OWASP top 10 for large language model applications. https://genai.owasp.org. Accessed 2026-10-14.

[16] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[17] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*, 2022.

[18] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[19] Sebastián Ramírez. FastAPI framework documentation. https://fastapi.tiangolo.com. Accessed 2026-10-14.

[20] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[21] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2302.04761.

[22] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.

[23] SQLite Consortium. SQLite documentation: Write-ahead logging. https://www.sqlite.org/wal.html. Accessed 2026-10-14.

[24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[25] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6): 186345, 2024. arXiv:2308.11432.

[26] Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. Feature hashing for large scale multitask learning. In *International Conference on Machine Learning (ICML)*, pages 1113–1120, 2009. arXiv:0902.2206.

[27] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.

[28] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.